

11.1.7 符号链接（软链接）

符号链接（Symbolic Link）是一种建立在文件路径层次之上的链接。符号链接文件本身是一个独立的文件，具有自己的文件名和 inode。但与普通文件不同，符号链接是一种特殊的文件^①，其数据内容是一个字符串，即该链接所指向的目标文件的路径。为了与传统的基于 inode 号的链接更好地区分，人们一般也将符号链接称为软链接（Soft Link），而将之前的链接称为硬链接（Hard Link）。

新建软链接时，用户仅仅传入一个字符串，操作系统会新建一个类型为软链接的 inode，并将字符串作为其数据。由于路径字符串的长度一般不会很长，因此文件系统可以做一个简单的优化：将路径字符串直接保存在 inode 中，占据原本用于保存数据块指针的空间，这样就可以不使用任何数据块，从而节约存储空间^②。当用户打开一个软链接文件时，文件系统会根据 inode 类型判断出这是一个软链接，于是会读取其保存的路径字符串，然后以该路径为参数再次查找对应的 inode，若没有找到则报错。因此，用户打开软链接文件，最终会打开其所指向的目标文件。从用户的角度来看，这个效果和硬链接是非常类似的。与硬链接不同的是，软链接和目标文件是两个不同的、独立的文件，可以保存在不同的文件系统中，因此可以实现跨文件系统的链接。

输出记录 11.2 展示了软链接的创建和其他操作示例。用户可通过带 `-s` 参数的 `ln` 命令创建一个软链接，其中 `-s` 后的字符串便是目标文件的路径。软链接和目标文件是独立的，拥有各自的 inode，因此即使目标文件不存在，软链接也可以存在，只是访问的时候会报错。软链接可以指向目录，这也是其与硬链接的不同之处。软链接的目标文件可以是另一个软链接，文件系统只需要再次根据路径查找即可。为避免出现回环导致无限深的路径，文件系统需要为路径的深度设置一个阈值，若超过该阈值则报错。

输出记录 11.2 使用 `ln -s` 命令创建符号链接

```
[user@osbook ~] $ ln -s a a.link
[user@osbook ~] $ cat a.link
cat: a.link: No such file or directory
[user@osbook ~] $ echo "hello, world" > a
[user@osbook ~] $ cat a.link
hello, world
[user@osbook ~] $ ls -l
-rw-r--r-- 1 osbook wheel 13 6 12 19:45 a
lrwxr-xr-x 1 osbook wheel 1 6 12 19:45 a.link -> a
[user@osbook ~] $ ls -li
90055602 a          90055547 a.link
[user@osbook ~] $ mkdir d
[user@osbook ~] $ ln -s d d.link
[user@osbook ~] $ touch d/x
```

① 回顾一下：目录文件也是一种特殊的文件，文件的类型记录在该文件 inode 的 mode 字段。

② 这个优化不仅可以用于软链接，而且可以用于其他很小的文件。

```
[user@osbook ~] $ ls d.link
x
[user@osbook ~] $ ln -s e e
[user@osbook ~] $ cd e
cd: too many levels of symbolic links: e
```

文件系统的设计空间非常广，inode 仅仅是其中的一种，现实中还有许多并非基于 inode 的文件系统。那么，不同的设计会为文件系统带来哪些能力上的不同呢？接下来我们介绍 Windows 系统中常用的两个文件系统——FAT 和 NTFS 的设计。

11.2 基于表的文件系统

本节主要知识点

- ❑ FAT 文件系统是如何保存数据的？
- ❑ NTFS 与 FAT 文件系统相比有哪些改变，带来了哪些好处？
- ❑ 用户态文件系统有哪些优势？如何在宏内核中提供用户态文件系统机制？

11.2.1 FAT 文件系统

文件分配表（File Allocation Table，FAT）^[14]是一种组织文件系统的架构。1977 年，Bill Gates 和 Marc McDonald 创建了第一个基于 FAT 的文件系统，用于管理 Microsoft Disk BASIC 系统中的存储设备^[5]。自被搭载在 Windows 操作系统中至今，基于 FAT 的文件系统从早期的 FAT12、FAT16 逐步发展到被广泛使用至今的 FAT32 和 exFAT 等文件系统。这些文件系统均使用 FAT 作为主要索引格式，后文中我们统一使用 FAT 文件系统来表示这一类文件系统。本节中，我们将以 FAT32 为基础介绍 FAT 文件系统的基本设计[Ⓐ]。

存储布局

图 11.8 展示了 FAT 文件系统的基本存储布局。位于整个文件系统最前端的是引导记录。引导记录的作用和超级块类似，其中记录了文件系统的元数据以及后续各个区域的位置。若此分区是可启动分区，则引导记录中还包括引导整个操作系统启动所需要的代码。

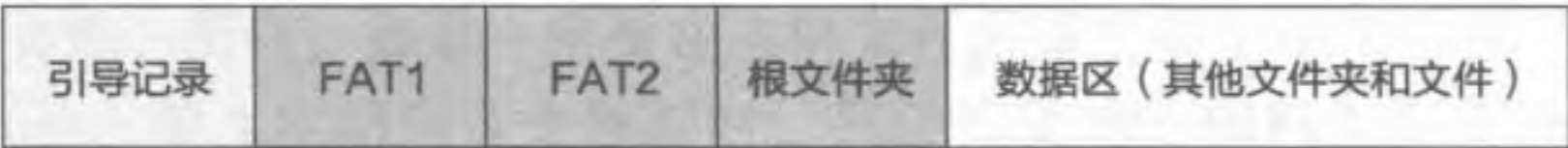


图 11.8 FAT 文件系统的存储布局^[2-3]

Ⓐ 由于 FAT 文件系统的版本众多，且在设计上会略有差别，实际使用中的 FAT 文件系统设计可能与本节的内容略有出入。

引导分区后通常为 FAT，FAT 通常有两份，在图 11.8 中分别标记为 FAT1 和 FAT2。两份 FAT 的内容一致，当 FAT1 由于各种原因损坏后，文件系统依然可以使用 FAT2 来访问和修复整个文件系统^①。

FAT 后通常保存根文件夹^②，然后为数据区，保存其他文件夹和文件数据。FAT 文件系统以簇 (Cluster) 作为逻辑存储单元^③，每个簇对应物理存储上的一个或多个连续的存储扇区 (Sector) (即存储设备的物理访问单元)。簇的具体大小可以在格式化时指定。

目录项格式

FAT 文件系统的目录中同样保存着目录项。与此前介绍的基于 inode 的文件系统不同，FAT 中每个目录项的大小固定为 32 字节，其中包括：

- 8.3 格式的文件名 (11 字节)
- 属性 (1 字节)
- 创建时间 (3 字节)
- 创建日期 (2 字节)
- 最后访问日期 (2 字节)
- 最后修改时间 (2 字节)
- 最后修改日期 (2 字节)
- 数据的起始簇号 (2 字节)
- 文件大小 (4 字节)
- 保留字节 (3 字节)

除了属性和 8.3 格式的文件名外，目录项中的其他内容从字面便可以理解。接下来我们详细介绍一下属性和 8.3 格式的文件名。

属性。属性用来表示该目录项的种类和状态。例如，属性中使用特定的位表示文件是否为只读、是否为隐藏、是否为系统文件、是否为子目录。为了支持长文件名，FAT 文件系统使用特殊值 0x0F 作为属性表示该目录项是一个长文件名的一部分，本节中称为长文件名构件。关于长文件名和长文件名构件，我们将在短文件名之后进行介绍。

小思考

此前介绍的 inode 文件系统中，并未在目录项中区分子文件和子目录 (子文件夹)。在 FAT 文件系统中，目录项中特意使用一位 (Subdirectory 位) 区分这两者。为什么 FAT 需要这样设计呢？

① 在基于 inode 的文件系统中，也会将重要结构 (如超级块) 保存多份，以防止重要结构的损坏影响整个文件系统的使用。

② 即根目录，在 Windows 中称前文介绍的目录为“文件夹”。

③ 对应前文中使用的“块”。

短文件名。短文件名（8.3 格式）是一种占用 11 字节的文件名格式，其中前 8 字节为文件名，后 3 字节为拓展名。因此，一个名为 datafile.dat 的文件使用 8.3 格式时，保存为 DATAFILEDAT 共 11 字节。由于点“.”的位置是固定的，因此不需要实际进行存储。如果文件名超过 8 字节，则会被截断并添加序号以做区别。如 mydatafile.dat 会被变为 MYDATA ~ 1.DAT，其中 MYDATA 为原文件名的前 6 个字符，~ 1 表示此文件名是被截断的，而且其是此类文件名中的第一个。如果 MYDATA ~ 1.DAT 文件已经存在，则 mydatafile.dat 会被变为 MYDATA ~ 2.DAT，以此类推。此外，8.3 格式中并不支持小写字母，所有文件名中的小写字母会被自动转换成大写字母进行存储。

长文件名。如何让文件支持更长的文件名呢？FAT32 文件系统的解决方法是在保留短文件名的同时，用多个额外的目录项来保存长文件名，从而弥补短文件名表达力的不足。因此如果一个文件拥有长文件名，那么它就同时拥有了两个文件名。应用程序可以使用其中任意一个来找到这个文件。

图 11.9 展示了 FAT 文件系统中的—个名为“OSBook File System.tex”的文件的长文件名存储。首先，文件系统会为此文件自动生成一个短文件名“OSBOOK ~ 1.TEX”，如图 11.9 下方所示。该文件的长文件名则使用连续的两个长文件名构件类型的目录项进行存储。这两个长文件名构件目录项保存在短文件名目录项之前（图 11.9 的上方）。



图 11.9 FAT 文件系统的长文件名 [2-3]

用于保存长文件名的目录项，其类型是 0x0F（如图 11.9 中粗线方框所示），用于与正常的目录项区分。若需要多个目录项才能保存下文件名，那么第一个目录项的编号是 0x1，第二个是 0x2，以此类推（如图 11.9 中深色方框所示）。注意最后目录项的序号需要与 0x40 进行异或操作，因此图 11.9 中的最后一个目录项的序号是 0x42。每个目录项除开头的序号、类型、预留的 0x00 和校验字段外，均可用于保存文件名。本例中，每个字母占 2 字节，这是因为 Windows 采用 Unicode 的编码方式，这样每个目录项可以最多保存 13 个 Unicode 字符。FAT32 限制文件名的长度不能超过 255 字节（小于 20 个目录项）。

FAT 和文件格式

前面我们提到 FAT 文件系统以簇作为逻辑存储单元。每个簇有一个地址，称为簇号。FAT 实际上是一个由簇号组成的数组——每个簇都有其对应的 FAT 表项，而每个 FAT 表项中记录了一个簇号。FAT 中记录的簇号为逻辑上下一个簇的簇号，举例来说，簇 3 的 FAT 表项中记录了 5，说明簇 3 之后的下一个簇是簇 5。十六进制全 F 表示当前簇已经是最后一个数据簇，不存在下一个簇。

图 11.10 展示了两个文件在使用 FAT 时的存储方式。为了简洁，图中的目录项只显示了文件名和开始簇号，其他信息并未显示。当文件系统拿到一个目录项时，会根据其中保存的开始簇号找到簇，并从中读取数据。如在 FILE1.DAT 对应的目录项里，标记开始簇号为 2，因此我们可以从簇 2 中找到文件的第一个簇里面的数据。若一个簇不足以保存所有的文件数据，文件系统会将当前簇号（例子中为 2）作为索引，从 FAT 中查找到下一个簇号。在图 11.10 中，FAT 的 2 号项保存的簇号为 4，说明簇 4 保存了该文件的下一部分数据（该文件第二个簇的数据）。同样，簇 4 对应的 4 号 FAT 表项记录的簇号为 5，因此簇 5 中保存了文件接下来的数据。簇 5 对应的 FAT 表项中保存的簇号为 0xFFFF，说明不存在下一个簇。对于图中另外一个文件 HW1.TXT，我们可以看出这个文件只占用了 1 个簇，即簇 3。

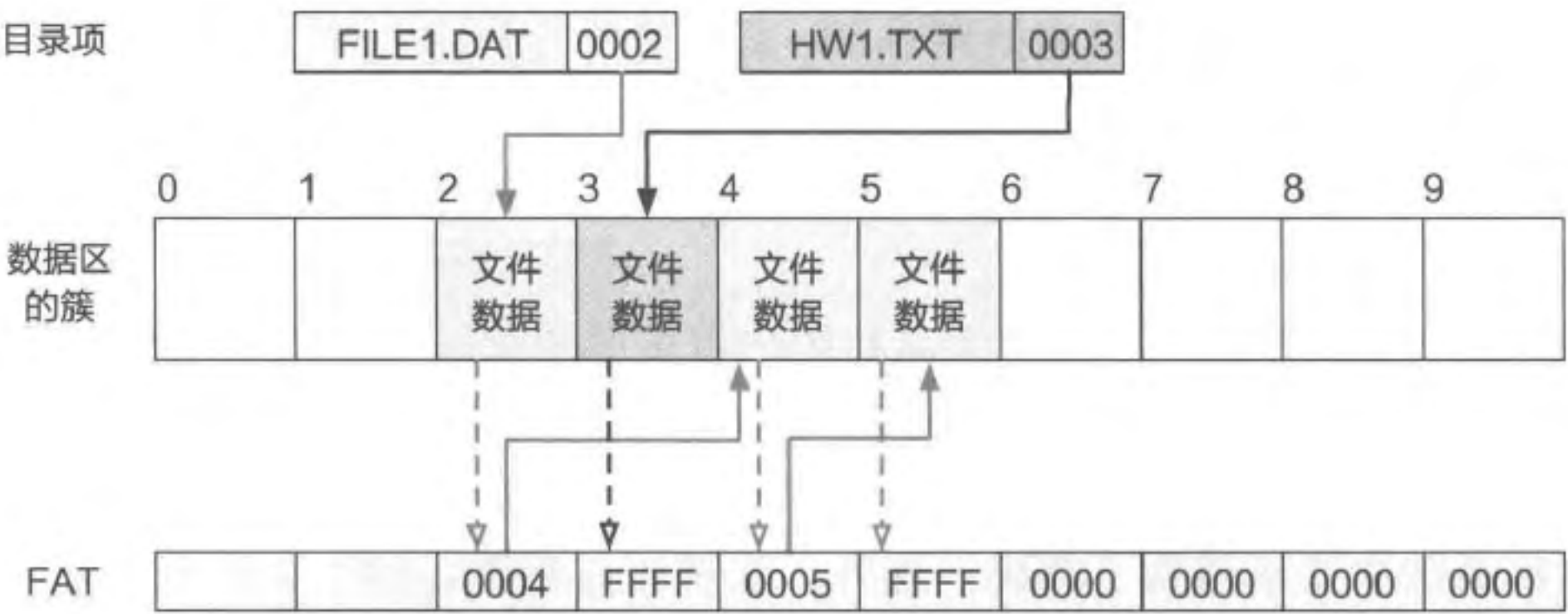


图 11.10 文件使用 FAT 格式进行保存^[3]

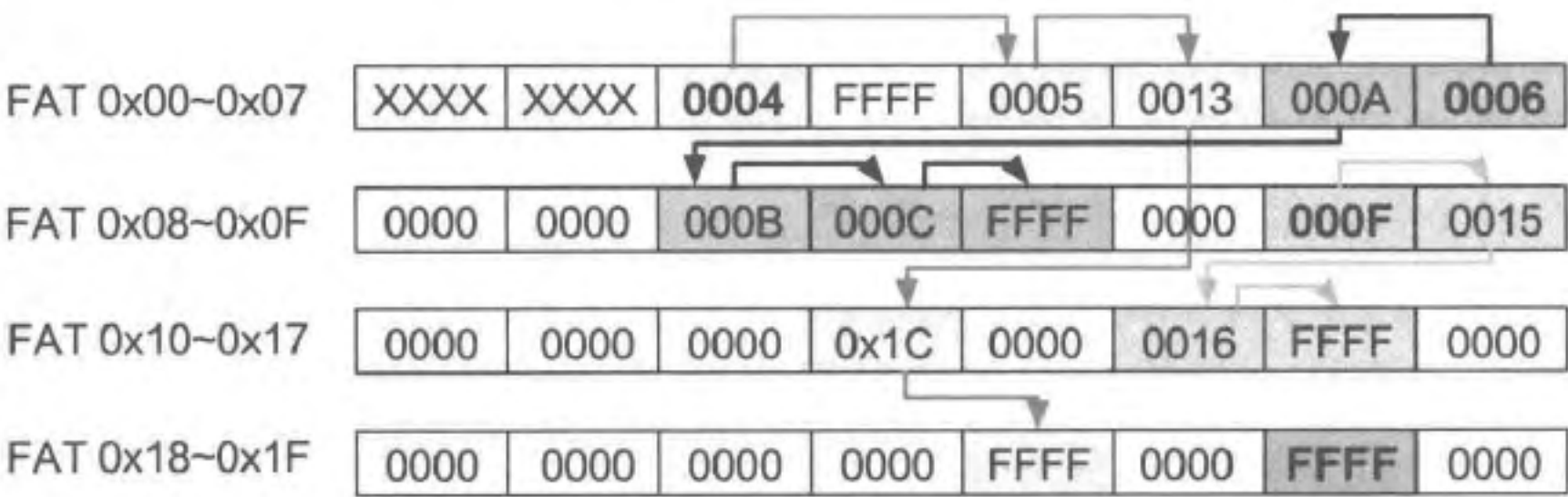
此处应该注意，图 11.10 为了便于理解，将目录项画在了最上方。但实际上，目录项也是保存在簇中的。

练习

11.3 为何 FAT 文件系统需要在目录项中区分文件和子目录？

为了更好地理解 FAT 的使用方法，我们在图 11.11 中给出了另外一个例子。图中给

出了一个 FAT 的前 32 项，以及 4 个文件（包括文件夹）的开始簇号。通过前述的方法，我们可以从 FAT 中得知每个文件的数据位置。



根文件夹开始簇号为2，其内容保存在簇2、4、5、0x13、0x1C中。
文件DOCUM.DOC开始簇号为7，其内容保存在簇7、6、0xA、0xB、0xC中。
文件夹MEDIA开始簇号为0xE，其内容保存在簇0xE、0xF、0x15、0x16中。
文件SCORE.TXT开始簇号为0x1E，其内容保存在簇0x1E中。

图 11.11 另外一个使用 FAT 的例子

FAT 文件系统小结

FAT 是 FAT 文件系统的核心。本质上，FAT 将一个文件的多个数据簇以链表形式进行索引。相对于我们此前介绍的使用 inode 索引文件数据的方法，FAT 的方法更加简单直接。然而 FAT 所使用的链式结构在查找时往往并不高效。如果要访问一个文件的中间部分，FAT 文件系统不得不逐个簇进行查找，使得大文件的访存变得尤其缓慢。这也是 NTFS 等后续文件系统出现的原因之一。

FAT 文件系统随着存储技术的提高和用户需求的变化不断演进。早期 FAT 文件系统的许多设计都进行了简化，如早期的 FAT 文件系统只支持 8.3 格式甚至 6.3 格式的短文件名，又如其在目录项中只保存了文件的访问日期，而没有访问时间。这些问题在后续的 FAT 文件系统中逐渐得到了解决。此外，在演进过程中，FAT 文件系统的簇号上限和簇大小不断提高，使其能够管理的总存储空间和单个文件大小也得到了提升，满足存储设备容量的增长和对大文件的使用需求。

由于设计和实现的简单性，FAT 文件系统依然被广泛用于各种设备和场景中。例如，UEFI 中 EFI 分区上的文件系统便是基于 FAT 文件系统进行设计的，并且与 FAT 文件系统兼容^[6]。

11.2.2 NTFS

NTFS (New Technology File System)^[4, 7-9] 是在 FAT 文件系统之后，另一个在 Windows 操作系统中广泛使用的文件系统。相对于此前的 FAT 文件系统，NTFS 在功能、性能、可靠性等方面进行了诸多提升，并解除了原有 FAT 文件系统中的诸多限制。

存储布局与 MFT

在 NTFS 中，文件系统的核心结构从 FAT 变成主文件表（Master File Table, MFT）。微软公司关于 NTFS 的文档^[8]将 MFT 称为一个数据库：MFT 中的每一行对应着一个文件，每一列为这个文件的某个元数据。NTFS 中所有的文件均在 MFT 中有记录。NTFS 中的所有被分配使用的空间均被某个文件所使用。即使是那些用于存放文件系统元数据的空间，也会属于某个保留的元数据文件。

图 11.12 展示了 NTFS 的存储布局 and MFT。从表面上来看，NTFS 的存储布局仅将 FAT 文件系统中的 FAT 变为 MFT，但实际上，MFT 结构与 FAT 有着本质区别。FAT 是一种固定的结构，其大小和完整位置在创建文件系统时就已经固定下来并记录在引导区域内。这样虽然简单高效，但同时也限制了文件系统的拓展性。而在 NTFS 中，并非所有的 MFT 记录位置在创建文件系统时就已经固定。MFT 的内容（即 MFT 记录）保存在一个名为 \$Mft 的元数据文件^①中。此文件的元数据被保存在 MFT 的第一条记录中。通过此种方法，MFT 能够“自己管理自己”，这类似于编程语言中的“自举”概念。在这种设计下，NTFS 的引导区域中只记录 MFT 的开头几条记录的位置，其中包括 \$Mft 文件本身的记录。通过 \$Mft 文件本身的记录，可以找到 \$Mft 文件所有的数据保存的位置，进而可以找到 MFT 中剩余的所有记录。

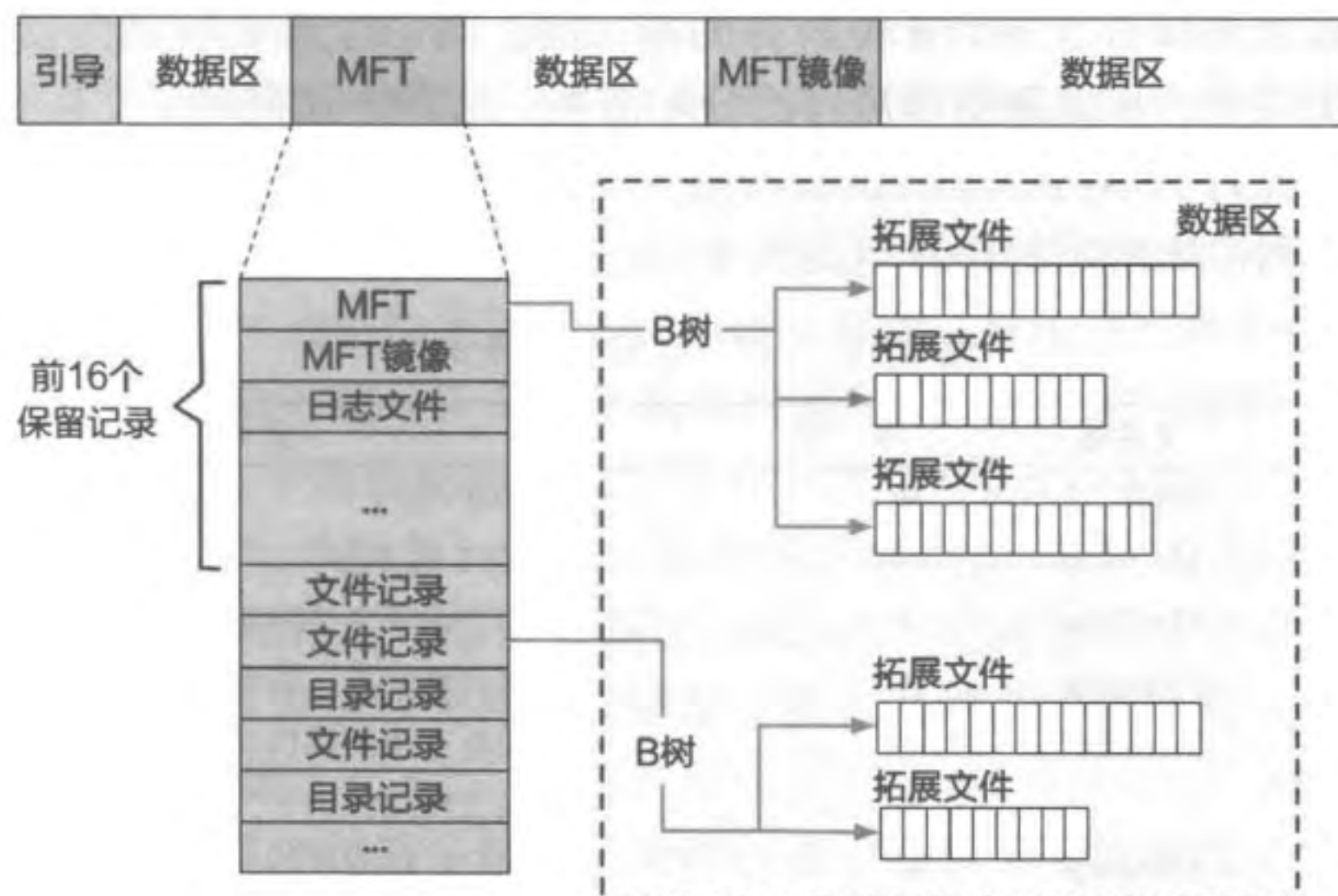


图 11.12 NTFS 的存储布局 and MFT^[9]

除了 MFT 中的前几条记录之外，其余记录可以分散存储在文件系统的各个位置，只要它们能够通过 \$Mft 记录中保存的索引找到即可。但考虑到 MFT 需要被频繁访问，如果其存储过于分散，将影响对 MFT 的访问效率。因此，NTFS 一般会预留整个文件系统存储空间的 12.5% 专门保存 MFT 的数据。只有在存储空间不足等特殊情况下，这些空

① 关于元数据文件，我们将在后文中进行具体介绍。

间才可以用于保存其他文件数据。

除了 MFT 之外，MFT 的镜像也与 FAT 文件系统中的 FAT 备份（即 FAT2）有所不同。在 FAT 文件系统中，FAT2 是 FAT1 内容的完整备份。而在 NTFS 中，MFT 的镜像只保存 MFT 文件的前四条记录[Ⓐ]。

MFT 表由多条 MFT 记录组成。其中前 16 条为保留记录，称为元数据文件。元数据文件的文件名一般以 \$ 开头，其中保存了文件系统的元数据。上文中，我们已经介绍了其中的 \$Mft 文件和 \$MftMirr 文件，分别表示 MFT 本身和 MFT 的镜像。表 11.3 列出了 MFT 中的全部元数据文件。从这些文件中，我们还可以看出一些 NTFS 与 FAT 文件系统相比增加的功能。

- NTFS 的元数据文件中包括一个日志文件，其以日志的形式临时保存文件系统的修改，以保证文件系统崩溃一致性。NTFS 的结构比 FAT 文件系统要复杂，因此在进行修改时更容易造成一致性问题。增加的日志文件能够让文件系统在系统崩溃重启后快速修复一致性问题。
- 属性定义文件定义了 NTFS 中所有属性的名字、数量和说明。NTFS 允许为每个文件增加额外的属性，从而增加了系统的灵活性，这在 FAT 文件系统中是不支持的。
- 如果存储设备上的某个簇已经损坏，再使用该簇保存数据会造成数据的损坏和丢失。坏簇文件保存了 NTFS 中所有的损坏簇。当一个簇被检测出损坏后，其会被加入此文件中，从而避免其被再次分配出去。这种方式提高了文件系统的可靠性。
- 拓展文件允许为 NTFS 提供扩展功能。一个拓展功能的例子是磁盘限额，其可以限制每个用户所使用的磁盘空间大小。

表 11.3 MFT 中的元数据文件

元数据文件	文件名	序 号	文件说明
MFT	\$Mft	0	保存 MFT
MFT 镜像	\$MftMirr	1	MFT 的备份
日志文件	\$LogFile	2	用于保证文件系统元数据一致性
卷	\$Volume	3	保存卷信息，如卷标和版本
属性定义	\$AttrDef	4	保存支持的属性名、数量和描述
根文件夹	.	5	根文件夹
簇的位图	\$Bitmap	6	标记已经使用的和空闲的簇
启动扇区	\$Boot	7	保存用于加载和启动卷的信息和代码
坏簇文件	\$BadClus	8	保存卷中损坏的簇
安全文件	\$Secure	9	保存卷中每个文件的唯一安全描述符
大写表	\$Upcase	10	用于进行 Unicode 相关转换
拓展文件	\$Extend	11	用于文件系统拓展，如磁盘限额
		12 ~ 15	未使用的记录

Ⓐ 如果前四条记录大小不足一个簇，则保存第一个簇中的所有记录。

MFT 记录的属性

MFT 中的每条记录可以包含多个文件属性。文件的所有信息均直接或间接地以属性的形式保存在 MFT 记录中，例如文件的时间戳保存在标准信息属性中，而文件数据保存在数据属性中。一些常用的属性包括：

- 标准信息：表示文件的基本元数据，包括访问模式（如只读或读写）、时间戳和链接数等标准信息。
- 属性列表：MFT 记录中未存储的其他属性记录的位置。
- 文件名：保存了该文件的文件名。文件名属性可以重复出现。长文件名允许使用最长 255 个 Unicode 字符。短文件名同 FAT32 一样，依然是 8.3 格式，且大小写不敏感。
- 数据：文件数据。
- 重解析点：用于挂载存储设备。
- 索引根：用于文件夹和其他索引。
- 索引分配：用于大文件夹和大文件的 B 树结构。
- 位图：用于大文件夹和大文件的 B 树结构。
- 卷信息：只用于 \$Volume 系统文件中标记卷的版本。

与 FAT 中的簇链表不同，NTFS 使用不同的方式保存不同大小的文件和目录。对于小文件和目录（通常不超过 900 字节），NTFS 将其内容使用数据属性保存在 MFT 记录中。此时的属性称为常驻（Resident）属性。若文件或目录的内容过多，无法在一个 MFT 记录中完整保存，NTFS 使用 B 树和区段^①的方法进行索引和保存。结合这两种方法，NTFS 既保证了小文件能够在 MFT 表中快速访问，又保证了在大文件中进行操作的高效性。

图 11.13 中给出了 MFT 记录以及其中属性的具体格式。每个 MFT 记录以一个 MFT 记录头开始，此后包含若干属性记录，最终以一个类型为 0xFFFFFFFF 的特殊属性结束。MFT 记录头中记录一些文件信息。例如，魔数用于表示该 MFT 记录的类型（如文件或目录），分配的字节数和使用的字节数分别表示该 MFT 记录占用的空间大小和其中已经使用的空间大小，链接数表示该文件的硬链接个数，属性偏移表示第一条属性记录距离该 MFT 记录开头的偏移量。

每条属性中记录了属性类型、属性记录的长度、是否为常驻属性、属性名字和名字的长度、属性内容的长度、属性内容距离属性记录开头的偏移量以及变长的属性内容等信息。属性类型为 0xFFFFFFFF 的属性记录表示 MFT 记录的结束，该属性记录只有属性类型这一个字段。例如，表示文件名的属性的属性类型为 0x30，属性内容为一个文件名结构，保存在属性记录的最后。文件名结构的具体格式如图 11.14 所示，其中包括该文件名的父目录的 MFT 记录号（相当于 inode 号）、各类时间、占用大小和实际使用大小、

① 一些连续的簇可组成一个区段。

文件名长度和具体文件名等。



图 11.13 MFT 记录以及属性

文件名结构	文件名长度和类型	文件名内容		
	实际使用的空间大小		文件属性标识	拓展属性
	最后一次MFT记录访问时间		分配的空间大小	
	最后一次属性内容修改时间		最后一次MFT记录修改时间	
	父目录的MFT记录号		创建时间	

图 11.14 文件名结构

目录项、访问时间与硬链接

图 11.15 中给出了 NTFS 中常驻目录的格式。当目录中的目录项不多，可以被保存在索引根属性中时，该目录被称为常驻目录。常驻目录的索引根属性包括索引根的头部信息和若干索引记录，即目录项。其中索引头部信息中记录了首个目录项距离索引头部开头的相对偏移量、索引的长度和占用空间等信息。通过这些信息可以访问此后保存的目录项。NTFS 的目录项中主要保存文件名和 MFT 记录号。文件名保存在一个完整的文件名结构中，MFT 记录号是指对应文件在 MFT 表中的编号，其作用与 inode 号相似。

除此之外，NTFS 目录项的文件名结构中还保存了目标文件的最后访问时间（Last Access Time）。NTFS 为每个文件维护了最后访问时间，表示该文件数据和元数据最后被访问的时间。NTFS 将每个文件的准确最后访问时间维护在内存中，只有当内存中的最后访问时间与存储设备上的最后访问时间相差了一个小时之后，才会将最后访问时间更新到存储设备上。对于每个文件，有两个位置保存了其最后访问时间。除了在目录项中外，最后访问时间还保存在文件的标准信息属性中，即 MFT 记录中的文件名属性中。



图 11.15 目录和目录项

基于目录项，NTFS 还增加了硬链接支持。每个硬链接拥有一个单独的目录项，但是其中保存的文件 ID 指向了相同的 MFT 记录。此时，该 MFT 记录中为每一个硬链接记录一个单独的文件名属性。

多数据流

文件数据是一个字符序列，又可以被称为一个数据流 (Data Stream)。NTFS 中的文件数据保存在默认的数据属性中，是文件的默认数据流。

除默认数据流之外，NTFS 还允许在同一个文件中保存多个数据流。从命名上来看，file.doc 表示该文件的默认数据流，而 file.doc:stream1 表示该文件中名为 stream1 的数据流。从存储上来看，每个数据流保存在一个单独的属性中，如默认数据流保存在默认的数据属性中，而 stream1 数据流保存在名为 stream1 的数据属性中。每个数据流有独自的一组文件属性，包括锁、流的大小等，因此多个数据流的访问之间互不干扰。多数据流允许应用程序将该文件相关的附带信息保存在该文件的特定数据流中，如图片编辑器可以将图片的缩略图保存在名为 thumbnail 的数据流中，又如编译器将程序的调试信息保存在名为 debuginfo 的数据流中。

压缩和稀疏文件

NTFS 允许对单个文件、目录中的所有文件以及整个卷 (即文件系统) 进行压缩保存。在启用压缩功能之后，存储设备上保存压缩后的数据。当应用程序访问 NTFS 上压缩保存的文件时，NTFS 会自动将被访问的那一部分文件数据解压缩，并将解压缩后的文件数据保存在内存中供给应用程序使用。该过程对应用程序来说是透明的。

除了压缩外，NTFS 还增加了对稀疏文件 (Sparse File) 的存储优化。当一个文件中存在大片连续数据为零数据 (即所有位均为 0) 时，其被称为稀疏文件。按照普通文件的方法来存储稀疏文件是很不划算的，因为其中的零数据会占用存储空间，却没有实际意义。为此，当一个文件被标记为稀疏文件时，NTFS 只保存其中的非零数据 (以及位置和长度)，其余的零数据并不保存在存储设备上。当应用程序对文件的零数据区域进行读取

时，NTFS 直接返回相应的零数据。

对于稀疏文件的优化提高了稀疏文件的存储效率和访问效率。考虑一个 10GB 的文件，其中只有开头 1GB 和结尾 1GB 中保存了有效数据，中间的 8GB 全都是零数据。如果按照普通的方法存储，则至少需要占用 10GB 存储资源，且对文件的访问需要访问复杂的文件索引找到对应的簇，再从其中读取数据。而按照优化后的方法存储，NTFS 只需要保存文件开头和结尾各 1GB 的数据，中间部分无须存储。当对中间部分进行访问时，NTFS 可以直接返回零数据，无须通过索引找到具体的簇。

11.3 虚拟文件系统

本节主要知识点

- 多个不同的文件系统如何共同工作在同一个操作系统之上？
- 为何应用程序能使用统一的接口访问多个不同文件系统上的文件？
- 伪文件系统是什么？为什么要有伪文件系统？

计算机系统中可能同时存在多种文件系统。例如，在同一台计算机中，用户分别使用 NTFS 和 Ext4 两个文件系统管理一块机械硬盘上的不同区域，用于保存电影和数据备份。同时，在一块固态硬盘中，用户使用 Ext4 文件系统保存系统文件和频繁使用的数据。当在不同的计算机系统之间传递数据时，用户还经常将 FAT32 文件系统格式的 USB 闪存盘接入计算机系统。

在操作系统中，虚拟文件系统（Virtual File System，VFS）^①对多种文件系统进行管理和协调，允许它们在同一个操作系统上共同工作。不同文件系统在存储设备上使用不同的数据结构和方法保存文件。为了让这些文件系统工作在同一个操作系统之上，VFS 定义了一系列内存数据结构，并要求底层不同的文件系统提供指定的方法，将其存储设备上的元数据统一转换为 VFS 的内存数据结构，VFS 通过这些内存数据结构向上为应用程序提供统一的文件系统服务。在本节中，我们将以 Linux 中的 VFS^[10]为例，介绍 VFS 如何管理和调用多种文件系统，以及 VFS 如何为应用程序提供统一的文件系统抽象。

11.3.1 文件系统的内存结构

在前两节中，我们主要介绍了文件系统在存储设备上的结构和操作。在实际使用的计算机中，数据需要首先被载入内存，才能被 CPU 访问和修改。因此，文件系统的正常运行离不开内存中与文件系统相关的结构。

① 又称 Virtual File system Switch。